

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчёт

Лабораторная работа 2

Выполнила:
Вдовина Анастасия
Группа J3210

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Целью данной лабораторной работы является изучение взаимодействия клиентской части веб-приложения с внешним API: получение данных с сервера, отправка данных на сервер и работа с закрытыми разделами, доступ к которым требует авторизации.

Темой проекта остаётся платформа для покупки билетов на мероприятия EventTix, позволяющая искать концерты и фестивали, покупать билеты и управлять ими в личном кабинете.

По условию задания требовалось реализовать моковое API средствами JSON-сервера и связать с ним страницы сайта, а также подключить к этому API авторизацию. Обращения к серверу выполнены с помощью библиотеки `axios`.

Содержимое сайта — мероприятия, пользователи, билеты, возвраты и отзывы — больше не хранится в разметке: страницы получают его с сервера и собирают интерфейс уже из полученных данных.

Ход работы

Работа выполнялась в несколько этапов: сначала были спроектированы данные и запущено моковое API, затем к нему поочерёдно подключались страницы каталога, мероприятия и личных кабинетов, после чего была добавлена авторизация и разграничен доступ к защищённым разделам. Каждый этап сопровождается краткой теоретической справкой по тому, что на нём применялось.

Этап 1 — проектирование данных и запуск мокового API

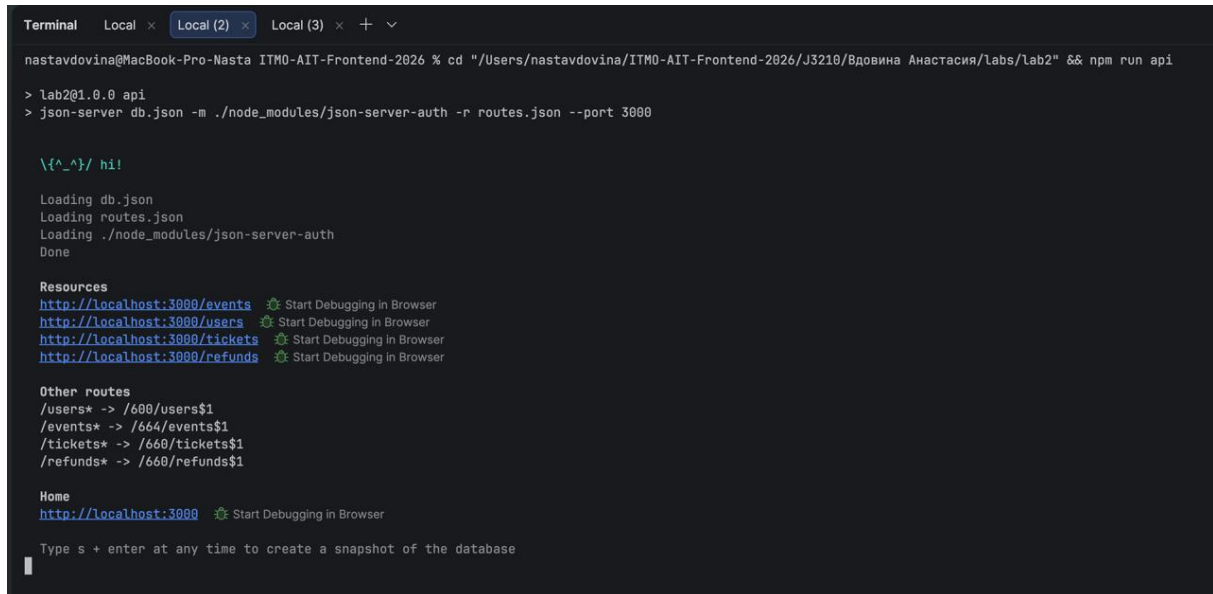
На первом этапе всё содержимое сайта было перенесено из разметки в файл `db.json`, который играет роль базы данных. В нём четыре коллекции: мероприятия, пользователи, билеты и возвраты. Каждое мероприятие описано полным набором полей — название, тип, город, дата и время, площадка, стоимость, описание, схема зала и список отзывов, — то есть всем, что раньше было записано в разметке страниц вручную.

Для запуска API использован пакет `json-server`: он читает `db.json` и автоматически поднимает набор HTTP-адресов для чтения и изменения каждой коллекции, не требуя ни строчки серверного кода. Сервер запускается командой `npm run api` и работает на порту 3000, сам сайт открывается отдельно как статические файлы.

Такое разделение — данные на сервере, интерфейс на клиенте — соответствует тому, как устроены настоящие веб-приложения: клиент ничего не знает о том, где и как хранятся данные, он лишь запрашивает их по известному адресу.

JSON как формат сериализации

Сериализация — это перевод структуры данных в последовательность символов, которую можно передать по сети или сохранить в файл. JSON — самый распространённый в вебе формат такой записи. Важно не путать JSON с объектом JavaScript: JSON — это *текст*, где ключи и строки обязательно в двойных кавычках, нет комментариев, нет висящих запятых и нет типа «дата» — даты передают строками. Обратное преобразование текста в объект называется десериализацией.



```
Terminal Local x Local (2) x Local (3) x + v
nastavdovina@MacBook-Pro-Nasta ITM0-AIT-Frontend-2026 % cd "/Users/nastavdovina/ITM0-AIT-Frontend-2026/J3218/8довина Анастасия/labs/lab2" && npm run api
> lab2@1.0.0 api
> json-server db.json -m ./node_modules/json-server-auth -r routes.json --port 3000

\{^_^\}/ hi!

Loading db.json
Loading routes.json
Loading ./node_modules/json-server-auth
Done

Resources
http://localhost:3000/events Start Debugging in Browser
http://localhost:3000/users Start Debugging in Browser
http://localhost:3000/tickets Start Debugging in Browser
http://localhost:3000/refunds Start Debugging in Browser

Other routes
/users* -> /600/users$1
/events* -> /664/events$1
/tickets* -> /660/tickets$1
/refunds* -> /660/refunds$1

Home
http://localhost:3000 Start Debugging in Browser

Type s + enter at any time to create a snapshot of the database
```

Рис. 1: Запущенное моковое API: json-server перечисляет адреса созданных коллекций и правила доступа к ним.

Этап 2 — вывод каталога мероприятий из API

На втором этапе главная страница и страница каталога были отвязаны от статической разметки. Вместо готовых карточек в HTML остался пустой контейнер, который заполняется после ответа сервера: страница запрашивает список мероприятий, получает массив объектов и для каждого собирает карточку с афишей, типом мероприятия, датой, местом и ценой.

Запрос выполняется асинхронно с помощью `axios` и синтаксиса `async/await`. Отдельное внимание уделено тому, что видит пользователь, пока ответа ещё нет: на странице показывается индикатор загрузки, после успешного ответа он сменяется карточками, а при неудаче — понятным сообщением о том, что сервер недоступен. Правильность этого поведения проверялась запуском сервера с искусственной задержкой ответа в одну секунду.

Названия мероприятий подставляются в карточку как текст, а не как часть HTML-строки: мероприятия может создавать организатор, то есть их названия — это пользовательский ввод.

Почему сетевой код асинхронный

Ответ сервера приходит не мгновенно, а браузер выполняет весь код страницы в одном потоке. Если бы программа останавливалась и ждала ответ, интерфейс замирал бы целиком — не работали бы ни кнопки, ни прокрутка. Поэтому сетевые функции сразу возвращают промис — объект-обещание будущего результата, — а обработка ответа откладывается на момент, когда он придёт. Слова `async` и `await` лишь делают такой код внешне похожим на последовательный, не превращая его в синхронный.

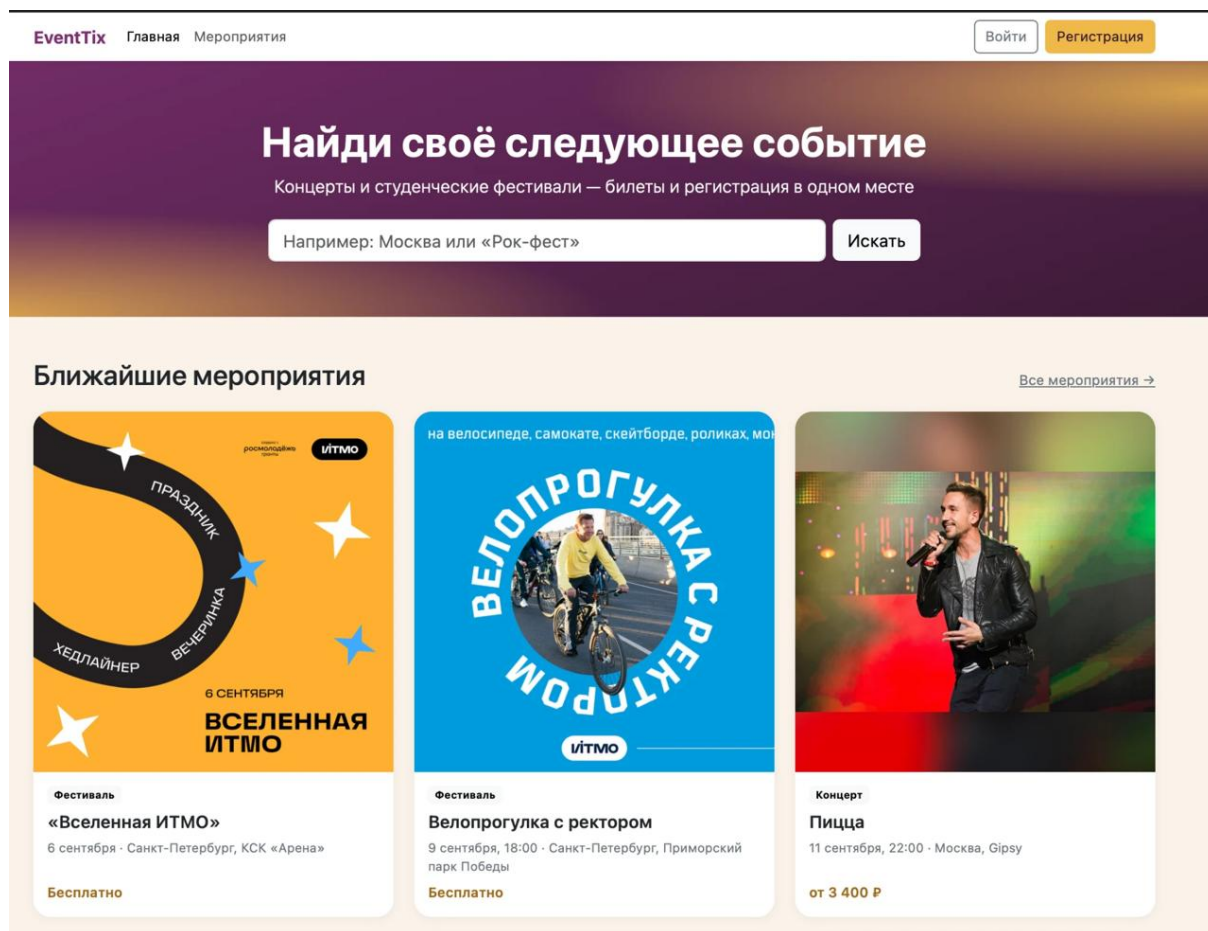


Рис. 2: Главная страница: карточки ближайших мероприятий собраны из данных, полученных с сервера.

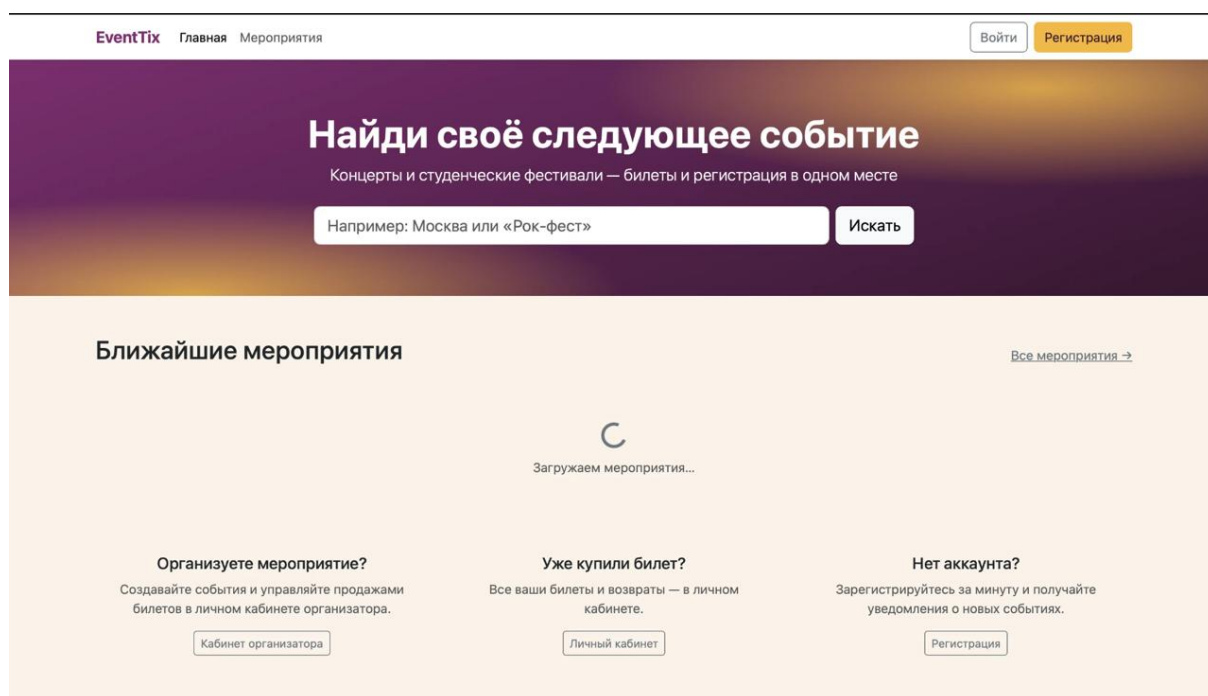


Рис. 3: Состояние загрузки: пока ответ сервера не получен, на месте карточек показывается индикатор.

Этап 3 — поиск и фильтрация силами сервера

На странице каталога есть поиск по названию и фильтры по типу мероприятия, городу и дате. На этом этапе они были переведены на работу с API: выбранные пользователем значения не отсеивают уже показанные карточки, а дописываются в адрес запроса, и сервер возвращает только подходящие записи.

Поиск по названию работает как полнотекстовый — сервер ищет введённую строку по всем полям записи, поэтому находятся мероприятия и по городу, и по площадке. Фильтр даты сформулирован как «дата от»: сервер возвращает всё, что происходит в выбранный день и позже. Поиск срабатывает прямо во время ввода, а запрос с главной страницы передаётся в каталог через адрес и подставляется в поле поиска.

Отдельно обрабатывается случай, когда под условия не подходит ни одно мероприятие: вместо пустой страницы показывается сообщение с предложением изменить параметры поиска. Это состояние отличается от ошибки связи — сообщения разные, потому что и причины разные.

Query-параметры

Всё, что идёт в адресе после знака вопроса, — это параметры запроса, пары «имя и значение». Они не указывают на другой ресурс, а уточняют, в каком виде вернуть уже известный: отфильтровать, отсортировать или разбить на страницы. Благодаря этому один и тот же адрес коллекции обслуживает и полный список, и любую выборку из него, а клиенту не приходится скачивать все данные, чтобы показать несколько записей.

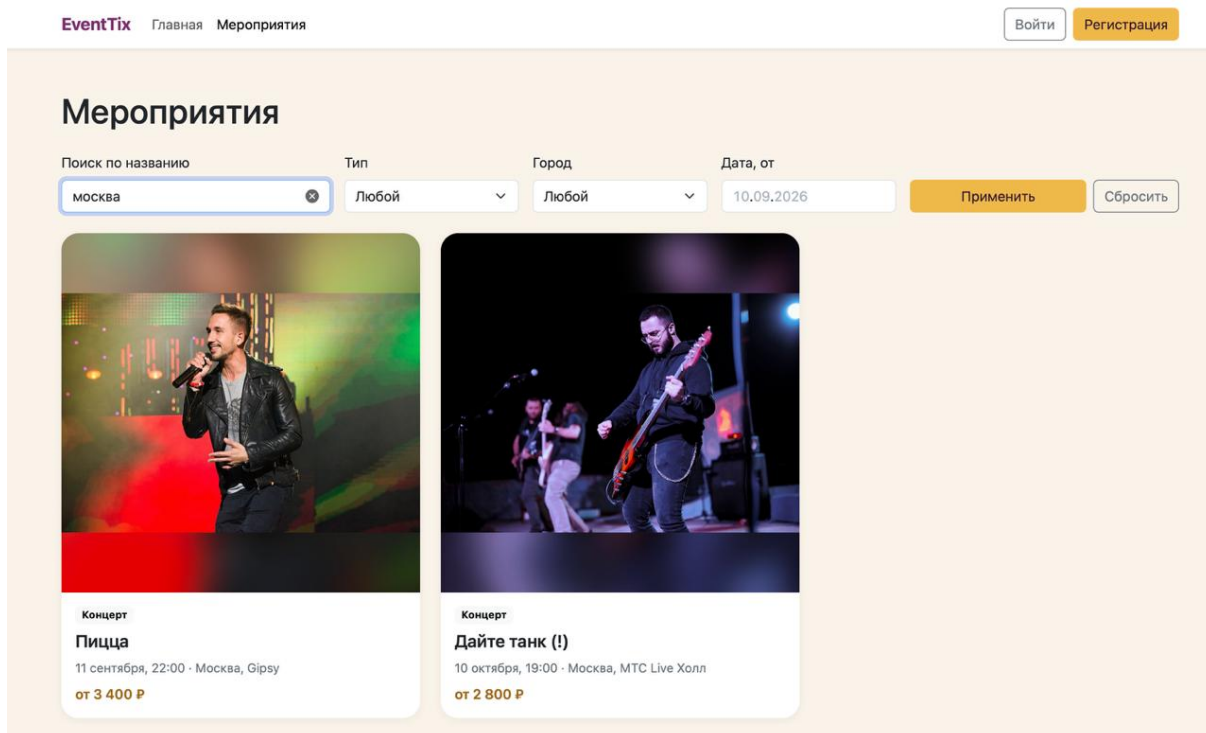


Рис. 4: Каталог мероприятий: выборка получена по параметрам поиска и фильтров, отправленным на сервер.

Этап 4 — страница мероприятия по идентификатору из адреса

Раньше каждому мероприятию соответствовал отдельный файл разметки. На этом этапе все они заменены одной страницей-шаблоном `event.html`, которая получает идентификатор мероприятия из адреса, запрашивает у сервера одну конкретную запись и заполняет собой афишу, описание, место проведения, схему зала, отзывы и блок покупки билета.

Такой подход сразу выявил вопрос: что делать с особенностями отдельных мероприятий. Например, на велопрогулке участника спрашивают, на чём он приедет, — раньше этот блок был просто вписан в разметку конкретной страницы. Теперь список вариантов хранится в данных мероприятия, а шаблон рисует выбор транспорта только тогда, когда такое поле у записи есть. Выбранный вариант затем попадает в билет как его категория.

Предусмотрены и неудачные случаи: если идентификатор не указан или мероприятия с таким идентификатором нет, страница не показывает пустой каркас, а выводит сообщение об ошибке.

Данные вместо разметки

Пока страниц немного, их можно писать руками, но каждое изменение оформления приходится повторять в каждом файле, и любая опечатка живёт в одной странице незаметно для остальных. Шаблон плюс данные решают обе проблемы: оформление описано один раз, а различия между страницами сведены к значениям полей. Побочный, но важный эффект — новые записи может добавлять сам пользователь через интерфейс, не редактируя ни одного файла разметки.

EventTix

ГлавнаяМероприятия

Войти

Регистрация

Главная / Мероприятия / Kanye West (Ye): Ye Live Tour



Билеты

от 17 000 Р

Количество

1

Итого от: 17000 Р

Купить билет

Концерт18+Ye Live Concert Tour 2026

Kanye West (Ye): Ye Live Tour

10 и 11 октября 2026, 20:00 · Санкт-Петербург, «Газпром Арена» (Футбольная аллея, 1)

Описание

Американский рэпер Kanye West (Ye) даст два больших концерта в Санкт-Петербурге в рамках мирового тура Ye Live Concert Tour 2026 в поддержку альбома Bully. Официальный организатор — концертное агентство Say Agency.

Категории билетов

Категория	Цена
Трибуны	от 17 000 Р до 50 000 Р
Танцпол	25 000 Р
VIP-ложи	до 160 000 Р

Внимание.

Осторожно, перекупщики. В сети и на платформах объявлений много мошенников, маскирующих чужие билеты под этот концерт. Покупайте билеты только на официальном сайте организатора yerussia2026.ru или в системе Intickets.

Место проведения

«Газпром Арена» — крупнейший стадион Санкт-Петербурга на Крестовском острове (Футбольная аллея, 1), вмещает более 60 000 зрителей.

Схема зала (сцена · танцпол · трибуны · VIP-ложи верхнего яруса)

Рис. 5: Страница мероприятия: все данные, включая категории билетов и предупреждение, получены с сервера по идентификатору из адреса.

Этап 5 — регистрация и вход

Пятый этап посвящён авторизации. К JSON-серверу подключён промежуточный слой `json-server-auth`, добавляющий к обычным коллекциям два служебных адреса — регистрации и входа.

Форма регистрации отправляет на сервер имя, электронную почту, пароль и

7

выбранный тип аккаунта. Сервер сам проверяет, не занят ли такой адрес почты, хэширует пароль и возвращает токен доступа вместе с данными пользователя — в файле данных пароль хранится уже в виде необратимого хэша, а не открытым текстом. Форма входа устроена так же, только проверяет уже существующего пользователя.

Полученный токен и данные пользователя сохраняются в локальном хранилище браузера, поэтому вход переживает перезагрузку страницы и переходы между страницами сайта. Ошибки сервера не остаются незамеченными: неверный пароль, несуществующий пользователь и занятая почта переводятся в понятные сообщения и показываются прямо в форме. Выход из аккаунта удаляет сохранённые данные.

После входа шапка сайта меняется на всех страницах: вместо кнопок «Войти» и «Регистрация» появляется меню с именем пользователя, а страницы входа и регистрации вошедшему больше не показываются — с них происходит переход в личный кабинет.

Аутентификация и авторизация

Аутентификация отвечает на вопрос «кто ты» — это проверка пароля. Авторизация отвечает на вопрос «что тебе разрешено» и происходит уже после того, как личность подтверждена. Результат аутентификации — токен: строка, которую сервер подписывает своим секретным ключом. Клиент не может её подделать, но и хранить её вечно нельзя — у токена есть срок жизни, после которого сервер перестаёт его принимать.

The screenshot shows the EventTix website's login page. At the top, there is a navigation bar with the EventTix logo, links for 'Главная' (Home) and 'Мероприятия' (Events), and two buttons: 'Войти' (Login) and 'Регистрация' (Registration). The main content area features a white login form titled 'Вход' (Login). The form contains a red error message: 'Неверный email или пароль.' (Incorrect email or password). Below this, there are input fields for 'Email' (containing 'anna@example.com') and 'Пароль' (password, masked with dots). A checkbox labeled 'Запомнить меня' (Remember me) is present. A yellow 'Войти' button is at the bottom of the form. Below the button, a test account is listed: 'Тестовый аккаунт: anna@example.com / password123'. At the very bottom of the form, there is a link: 'Нет аккаунта? [Зарегистрироваться](#)'. The footer of the page includes the copyright notice '© 2026 EventTix' on the left and a 'Наверх' (Back to top) link on the right.

Рис. 6: Форма входа: сервер отклонил неверный пароль, и ответ показан пользователю сообщением об ошибке.

Этап 6 — разграничение доступа

Наличие входа само по себе ничего не защищает, поэтому на шестом этапе для каждой коллекции были заданы права доступа. Каталог мероприятий читают все, включая незарегистрированных посетителей, но изменять его могут только вошедшие пользователи. Билеты и возвраты недоступны без входа полностью — ни на чтение, ни на запись. Данные учётной записи доступны только её владельцу.

Чтобы сервер узнавал пользователя, токен должен приходить с каждым запросом. Это решено перехватчиком: он автоматически добавляет токен в заголовок любого исходящего запроса, так что дописывать его вручную в каждой функции не нужно. Второй перехватчик следит за ответами: если сервер отвечает отказом из-за отсутствующего или истёкшего токена, сохранённые данные очищаются, а пользователь отправляется на страницу входа — вместо того чтобы молча смотреть на пустой экран.

Дополнительно закрыты сами страницы: личный кабинет и кабинет организатора не открываются без входа, а кабинет организатора доступен только пользователям с соответствующей ролью — посетитель, набравший его адрес вручную, попадёт в свой личный кабинет. Ссылка на кабинет организатора в меню посетителю не показывается.

Проверка показала, что защита работает именно на сервере: попытка создать мероприятие или дописать отзыв в обход интерфейса, без токена, отклоняется с ответом об отсутствии заголовка авторизации.

Клиентская проверка — это удобство

Скрытая кнопка и переадресация с закрытой страницы улучшают впечатление от интерфейса, но не защищают данные: код страницы выполняется на компьютере пользователя, и его можно изменить прямо в браузере. Единственная настоящая преграда — проверка на сервере, который решает, выдавать данные или ответить отказом. Поэтому клиентские проверки делают ради удобства, а не вместо серверных.

Этап 7 — запись данных: билеты, возвраты, отзывы и новые мероприятия

На последнем этапе интерфейс научился не только читать данные, но и изменять их. Покупка билета и бесплатная запись создают на сервере новую запись билета, привязанную к вошедшему пользователю, и она сразу появляется в личном кабинете.

Возврат билета устроен не как удаление, а как изменение: билет остаётся в списке со статусом «Возвращён», а в разделе возвратов появляется отдельная запись с датой и суммой. Так у пользователя сохраняется история — из списка ничего не исчезает бесследно.

Отзывы тоже перестали быть украшением: оставленный отзыв сохраняется в запи-

си мероприятия и виден после перезагрузки страницы. Поскольку изменять каталог разрешено только вошедшим пользователям, гостю вместо формы предлагается войти, а вошедшему имя подставляется из его аккаунта. Имя и текст отзыва попадают на страницу как текст: пользователь может ввести что угодно, в том числе разметку.

Кабинет организатора получил рабочую форму создания мероприятия: новое событие отправляется на сервер и тут же появляется и в кабинете, и в общем каталоге. Показатели кабинета — число мероприятий, проданных билетов, выручка и количество возвратов — считаются по актуальным данным, причём возвращённые билеты из выручки исключаются. Три нужных для этого запроса отправляются одновременно, а не один за другим.

HTTP-методы

Обращения к API различаются не только адресом, но и методом, и метод сообщает о намерении: получить данные, создать новую запись, заменить её целиком, изменить часть полей или удалить. Выбор метода — это не формальность: возврат билета меняет одно поле существующей записи, поэтому это частичное изменение, а не удаление. Удаление означало бы, что билета никогда не было, и история покупок пользователя оказалась бы неполной.

EventTix Главная Мероприятия Анна Смирнова ▾

Личный кабинет

Мои билеты Возвраты Данные аккаунта

Мероприятие	Дата	Тип билета	Статус	Действия
Kanye West (Ye): Ye Live Tour	10 и 11 октября 2026	Танцпол	Возвращён	—
«Вселенная ИТМО»	6 сентября 2026	Бесплатная регистрация	Активен	—
Велопрогулка с ректором	9 сентября 2026	Велосипед (свой)	Активен	—

© 2026 EventTix Наверх

Рис. 7: Личный кабинет: билеты и их статусы получены с сервера, возврат оформляется без перезагрузки страницы.

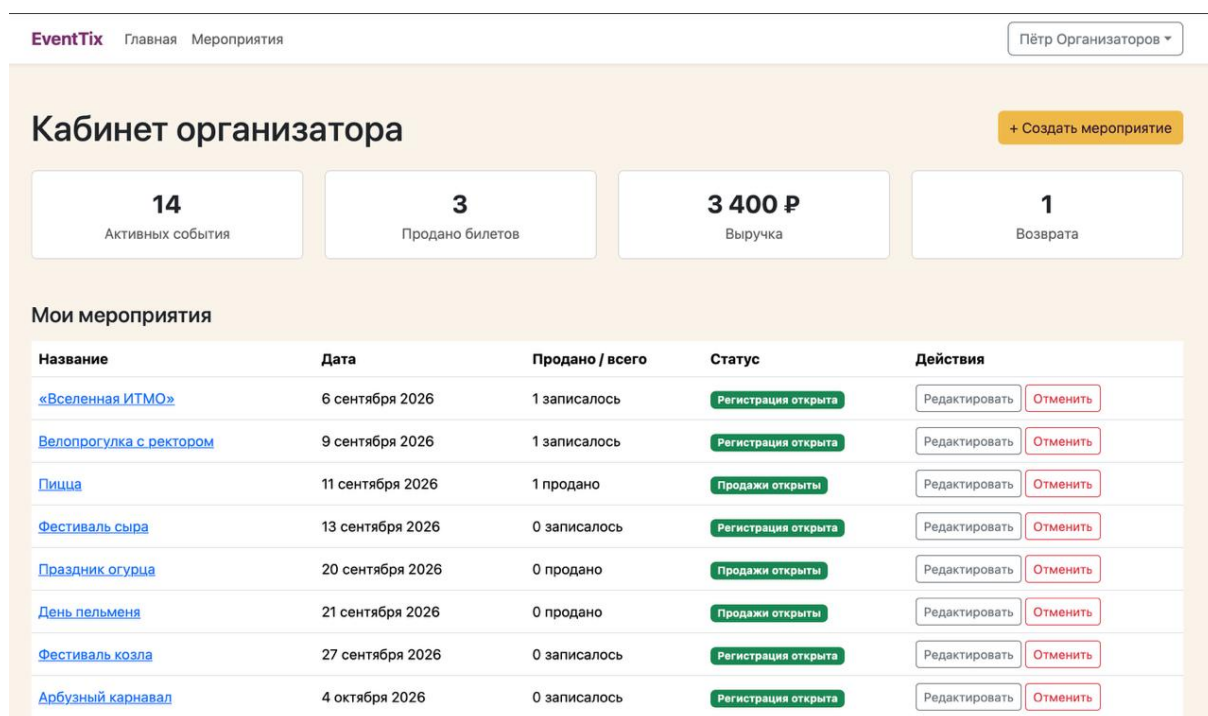


Рис. 8: Кабинет организатора: список мероприятий и показатели продаж рассчитаны по данным, полученным с сервера.

Вывод

В ходе выполнения лабораторной работы я изучила взаимодействие клиентской части веб-приложения с внешним API и научилась подключать к нему авторизацию.

Моковое API было реализовано средствами JSON-сервера, а все страницы платформы EventTix переведены на работу с ним: каталог, страница мероприятия, личный кабинет и кабинет организатора получают содержимое запросами к серверу, а формы покупки билета, возврата, отзыва и создания мероприятия отправляют данные обратно.

Отдельное внимание я уделила состояниям интерфейса. Любой запрос может идти долго или завершиться неудачей, поэтому каждая страница показывает индикатор загрузки, а при ошибке — сообщение о том, что произошло, вместо пустого экрана.

Работа с закрытой частью API показала разницу между внешним ограничением и настоящей защитой. Скрытые пункты меню и переадресация с закрытых страниц делают интерфейс понятнее, но доступ к данным определяет сервер: без токена он отклоняет запрос независимо от того, что делает клиент.

Полученное приложение остаётся учебным: данные хранятся в одном файле, а сервер предназначен только для локальной разработки. Тем не менее с точки зрения клиентской части оно ничем не отличается от работы с настоящим сервером — при замене адреса API код страниц менять не потребуется.